Decision Making under Uncertainty - Spring 2026
Assignment

January 25, 2026

General Rules

- The Assignment is divided into two parts: Tasks 1-2 are Part A, while Tasks 3-8 are Part B. In Part B, you can be with the same group or with a different group than the one you were with in Part A.
- Cooperation between members of different Groups is not allowed.
- The content of your report must be fully consistent with the code you submit. Implementing one thing and reporting another is considered *falsification/fabrication of results*, which constitutes very serious academic misconduct.
- All Group members are expected to contribute to all Tasks. Every member is collectively responsible for the correctness and quality of every Task.
- Any questions related to the Assignment must be directed exclusively to the course responsible. The Teaching Assistants are available only for exercise support and will not answer questions about the Assignment.
- Group Grade Determination: each Task i receives a grade $g_i \in [0, 100]$ and is weighted by w_i as specified. For part A, the overall Group grade G_A is given by the weighted average $\sum_{i\in[1,2]} w_i g_i$. For part B, the overall Group grade G_B is given by the weighted average $\sum_{i\in[3,8]} w_i g_i$ **<u>plus</u>** a bonus $0.1 \min_{\text{over } i\in[3,8]} g_i$ based on how well you did on your least good Task. The bonus is there to incentivize all members to tend to every Task, rather than dividing the assignment and working on separate Tasks individually.
- Individual Grade Determination: In the last Task of each Part p, you need to agree and jointly provide a percentage $f_{p,j} \in [0, 1]$ representing the contribution of each member j to the *whole* Part's work. It has to be $\sum_j f_{p,j} \leq 1$. Each member needs to have $f_{p,j} \geq 0.1$ to pass the course. The individual grade of a member j is then given by $\texttt{grade}_j = \sum_{p\in\{A,B\}} f_{p,j} \cdot N \cdot G_p$, where G_p is the group grade of part p as defined above, and N is the number of members in the group that have $f_{p,j} \geq 0.1$.

Problem Description

You are designing the operation of a restaurant's Heating and Ventilation system, to control the temperature and humidity. Decisions are made on an hourly basis, and the restaurant is open for 10 hours a day. The restaurant comprises two rooms. Each room r of which is equipped with

- an electric heater that can modulate its heating power continuously within given limits $[0, \overline{\mathrm{P}}_r]$;
- sensors that measure indoor temperature and indoor humidity.

The system also includes a central ventilation unit that can be switched ON or OFF, consuming a fixed amount of power $\mathrm{P}^{\texttt{vent}}$ when ON.

At each hour of operation, you get to decide the power consumption $p_{r,t}$ of each heater and whether the ventilation is ON (v_t). The owner wants to minimize the total electricity cost of the restaurant throughout a day of operation. The electricity price $\widetilde{\lambda}_t$ is different at each hour.

The temperature of each room at an hour t is modeled as a linear function of the following:

- the room temperature at $t - 1$,

- the temperature difference between the rooms, due to thermal exchange, at $t-1$ (coefficient $\zeta^{\texttt{exch}}$)
- the temperature difference between the room and the outdoors environment at $t-1$, due to heat losses through walls (coefficient $\zeta^{\texttt{loss}}$); the outdoors temperature $\mathrm{T}_t^{\texttt{out}}$ is assumed known in advance,
- the power of the room's heater at $t-1$ (coefficient $\zeta^{\texttt{conv}}$),
- the ventilation status at $t-1$ (coefficient $\zeta^{\texttt{cool}}$); if ON it decreases the room temperature,
- the room's occupancy level at $t-1$ (coefficient $\zeta^{\texttt{occ}}$) since more people increase the temperature.

Similarly, the humidity level of the whole place at a given hour t is modeled as a linear function of:

- the humidity level at $t-1$ (coefficient 1),
- the room's occupancy level at $t-1$ (coefficient $\eta^{\texttt{occ}}$), since more people increase the humidity,
- the ventilation status at $t-1$ (coefficient $\eta^{\texttt{vent}}$); if ON, it decreases humidity.

Finally, the owner has informed you of the specifics of the underlying control system:

<u>Temperature Overrule Controller:</u>
If the temperature of a room at an hour t drops below a given lower threshold $\mathrm{T}^{\texttt{low}}$, the system's overrule controller kicks in and, starting at t, forces that room's heater to operate at maximum power for as long as needed until the temperature surpasses a given "OK" threshold $\mathrm{T}^{\texttt{OK}}$. You cannot control the power when the overrule controller is active.
If the temperature of a room at an hour t rises above a given upper threshold $\mathrm{T}^{\texttt{High}}$, the system's overrule controller kicks in and forces that room's heater to operate at zero power for hour t. You cannot control the power when the overrule controller is active.

<u>Humidity Overrule Controller:</u>
If the indoor humidity at an hour t exceeds a given upper threshold $\mathrm{H}^{\texttt{high}}$, the ventilation system is forced to the ON state at t.

<u>Ventilation System Inertia:</u>
If the ventilation system is started at an hour t (i.e. it was OFF at $t-1$ and it is ON at t), it cannot be turned off for 3 hours (i.e. it remains ON throughout $t, t+1$ and $t+2$).

Your job is to design a policy for deciding the heating and ventilation actions at each hour. The objective is to minimize the total electricity cost (from heating and ventilation). Numerical values for all relevant parameters (power

consumption bounds of heaters, power consumption of ventilation, coefficients for each term of the temperature and humidity evolution functions, temperature and humidity thresholds, outdoors temperature) are given to you in the file `SystemCharacteristics`. The electricity price and room occupancy for an hour t is assumed to become known at the beginning of hour t. However, future electricity prices and room occupancy levels are uncertain.

Data for electricity prices and room occupancy levels:
You are given access to historical data of 100 days of operation. The files `PriceData`, `OccupancyRoom1` and `OccupancyRoom2` contain the electricity price and each room's occupancy level for each hour of each of 100 days.

PART A

Task 1: Optimal in Hindsight solution ($w = 0.05$)

Assume all uncertainties (occupancy level of each room and electricity price) are perfectly known beforehand for the whole day. Formulate and implement the Optimal in Hindsight solution as a mixed-integer linear program (MILP). The MILP should determine the optimal heating and ventilation actions for each hour of the day.

Deliverables:

1. pdf[1] describing the MILP in math mode. Follow the following template: Begin by defining the notation (nomenclature). Use calligraphic fonts for sets, italics for variables and indices, and straight or Greek letters for known quantities or coefficients. Specify clearly which are the problem's variables. Then, define the constraints and the objective function. Explain each constraint in words, in a single sentence, just before the constraint. Be concise and avoid wordy explanations.

2. Python code that implements this MILP. Use the file `SystemCharacteristics` to define numerical values for the problem's parameters. Solve the MILP for each of the 100 days of prices' and occupancy levels' data provided in the files `PriceData`, `OccupancyRoom1` and `OccupancyRoom2`. Calculate the average daily electricity cost.

3. pdf reporting the average daily electricity cost calculated. Also, plot the MILP problem's results for 2 example days of your choice, using the file `PlotsRestaurant`. Present the two figures (one for each day, containing the subfigures of the `PlotsRestaurant` template) and briefly interpret the results.

Task 2: Define the Problem ($w = 0.05$)

Formulate the problem as a Markov Decision Process. Assume that:

- the electricity price at hour t is given as a function of the electricity prices at hours $t-1$ and $t-2$;
- each room's occupancy level at hour t is given as a function of its own occupancy level at hour $t-1$ and the other room's occupancy level at hour $t-1$ (i.e. there is some correlation between the two rooms' occupancies).

Deliverable: pdf describing the MDP in math mode: Define the state variables, the decision variables, the reward function, and the transition dynamics (if for some state variable the transition dynamics is unknown, just state so). Use calligraphic fonts for sets, italics for states, decisions and indices, and straight letters for known quantities or parameters. Be concise and precise. Avoid overly wordy explanations.

Credit Allocation Task for Tasks 1-2

Report how each group member contributed to Tasks 1-2.

Deliverable: pdf where

- Each group member writes their name and one small paragraph about their own contributions to each Task. Be specific and comprehensive. The group reads, reviews, and approves each member's contributions. Take note that a false statement (for yourself or for others) is a breach of academic integrity.

[1] Include all pdf-deliverables of both Tasks in a single pdf file. Make sure to include the Group number and the names of all members in the first page.

- The group agrees and jointly provides a number f_i, representing each member's percentage contribution to the overall hand-in (not per Task) - see the General Rules at the top.

  Nuclear option: If you cannot reach an agreement, you can ask for a meeting with the instructor (to be scheduled after your deliverables are evaluated) where each member's contributions and overall understanding of the concepts will be assessed. Use this option only as a last resort; note that it **can** result in reduced points for all members if the assessment reveals weaknesses not discovered through your deliverables alone.

Deadline for Part A: *March 2nd, 2026 23.59.*

PART B

General Instructions for Tasks 3-5

In Tasks 3-5, you are to design some decision-making policies. Your policies will be evaluated in our own version of the environment, as if they were applied in the real system. Thereby, the time-series for electricity price and room occupancies will be different than the ones provided in the files and they are not known to you. However, a colleague of yours has already modeled the underlying process of the electricity price and room occupancies and has given the models to you; you can find them in the files `PriceProcessRestaurant` and `OccupancyProcessRestaurant`, which you are free to use. The time series on which your policies will be tested will be sampled from these processes.

For each of your policies, an average policy cost will be calculated by us, which essentially evaluates how good your policy is. To enable the execution of your policies in our environment, you should provide each policy as a python function using the template `PolicyRestaurant`[2]; do not change the "def" line (which defines the function's input from the environment) and also do not change the "return" line (which defines the here-and-now decisions to be returned by your policy to the environment). Optimization problems must be based on the Pyomo library and the solver needs to be Gurobi. Any library you might be using you need to import it.

One execution of each policy code (i.e. at a given hour of a given day) should take around 5-8 seconds. If it takes more than 15 seconds all actions are disregarded and the actions of the dummy policy are used instead in that hour. Also, if the policy returns actions that are outside the feasible bounds provided in the file `SystemCharacteristics`, the actions are mapped back to the nearest bound. If, for some reason, your policy returns faulty values (e.g. non-numerical), the actions are driven by the dummy policy instead. These checks will be performed using the code that you can find in the file `Checks`. **Make sure to run your policy by those checks before handing it in.**

Task 3: Stochastic Programming ($w = 0.2$)

Formulate, implement, and evaluate a multi-stage stochastic programming policy for the problem.

Deliverables:

1. pdf[3] presenting the details of the policy: number of lookahead timeslots, number of scenarios (initial and reduced) and structure (number of branches at each stage etc), and the stochastic optimization problem (sets, variables, constraints, objective function).

2. One .py file named "SP_policy_[your group's number]" containing the Python code with the policy - see instructions above under "General Instructions for Tasks 3-5" about how this will be evaluated.

Task 4: Approximate Dynamic Programming ($w = 0.2$)

Formulate, implement, and evaluate an approximate dynamic programming (ADP) policy for the problem, using a linear value function approximation.

Deliverables:

[2] Files `PolicyRestaurant` and `Checks` will be uploaded on Learn only after the deadline for Part A, because they reveal some information for the Tasks of Part A.

[3] Include all pdf-deliverables of all Part B Tasks in a single pdf file. Make sure to include the Group number and the names of all members in the first page.

1. pdf presenting the policy, including the approximate value function along with its training/configuration process, as well as how exactly it is used to make a decision.

2. One .py file named "ADP_policy_[your group's number]" containing the Python code with the policy - see instructions under section "Tasks 3-5: Designing Decision-making Policies" above.

Task 5: Hybrid Policy ($w = 0.2$)

This is your ultimate policy. You can use *any* type of policy e.g. a hybrid among stochastic programming, ADP, Policy Function Approximation, Cost Function Approximation.
Deliverables:

1. pdf explaining your ultimate policy, as briefly as possible but such that anyone can reproduce it without looking at your code. If part (or all) of it is the same as Task 2 or 3, you do not need to copy and paste.

2. One .py file named "Hybrid_policy_[your group's number]" containing the Python code with the policy - see instructions under section "Tasks 3-5: Designing Decision-making Policies" above.

Task 6: Your own Evaluation ($w = 0.2$)

Create a simulation environment in Python in which decision-making policies can be evaluated.
Deliverables:

1. Python code of the Environment: it takes any decision-making policy for this problem as input, and evaluates its average performance (output) in E independent Experiments (days). Use the file `SystemCharacteristics` to define numerical values for the problem's parameters and the files `PriceData`, `OccupancyRoom1` and `OccupancyRoom2` to draw time-series for prices and room occupancy levels for 100 different days/experiments.

2. Python code implementing a dummy policy that never turns on the ventilation nor any heater, and leaves everything up to the overrule controllers.

3. Python code implementing the Optimal in Hindsight Solution of Task 1.

4. pdf presenting and comparing i) the average daily cost and ii) the histogram of the daily cost, across the 100 experiments provided in the files `PriceData`, `OccupancyRoom1`, and `OccupancyRoom2` for the following policies:

   - the dummy policy,
   - the optimal in hindsight solution of Task 1,
   - your stochastic programming policy of Task 3,
   - the deterministic lookahead aka "expected value policy" i.e. a special case that considers a single (the expected) scenario,
   - a two-stage stochastic programming policy,
   - your ADP policy of Task 4,
   - your Hybrid policy of Task 5,

   as evaluated in your environment. Briefly interpret the results.

Task 7: Distributed Decision-Making ($w = 0.1$)

The restaurant is part of a mall that contains 14 more similar stores, i.e. a total of $N = 15$ stores. Each store is similar to the restaurant (two rooms, two heaters, etc). Since you did a great job with the restaurant, you are now hired to handle the energy management for the entire mall! The mall as a whole can only consume up to $\mathrm{P}^{\texttt{mall}}$ kWh of electricity at each hour, i.e.:

$$\sum_{n \in [1,N]} p_{n,t} \leq \mathrm{P}^{\texttt{mall}}, \quad \forall t \in \mathcal{T}. \tag{1}$$

Your objective is to satisfy this constraint, while minimizing the weighted squared distances of the stores from their desired temperature $\mathrm{T}^{\mathtt{ref}}$, as in:

$$\min \Big\{ \sum_{n \in [1,N]} \sum_{t \in \mathcal{T}} w_n \cdot (T_{n,t} - \mathrm{T}^{\mathtt{ref}})^2 \Big\} \tag{2}$$

For simplicity, assume that:

- There is no uncertainty: occupancy levels for the whole day are known in advance and are the same for each store (they are given in the file `Task7Occupancies.csv`.
- Each store/agent features no binary variables: the ventilation is always ON, the humidity constraints can be disregarded, and there are no overrule controllers.

The values for $\mathrm{T}^{\mathtt{ref}}, \mathrm{P}^{\mathtt{mall}}$ and all the other necessary data can be found in the file `DataTask7`. For each store $n \in [1, N]$, set $w_n = n + 1$. Design a distributed optimization algorithm for coordinating the stores' energy consumption.

Deliverables:

1. pdf describing the distributed algorithm in pseudocode. Give enough information so that your code and results can be reproduced by anyone without actually looking at your code;
2. pdf presenting your simulation results. Execute the algorithm for the whole day and present:
   - The system's objective value (Eq.2 above) in the y axis across 100 iterations of the algorithm (x axis) for five different choices of the Lagrange multipliers' step size parameter α (five curves in the same figure for $\alpha = \{0.001, 0.01, 0.1, 1, 10\}$). In the figure, also note the objective value at the optimal solution (obtained by solving one centralized optimization problem with all stores) using a horizontal line. Do the same for an adaptive step $\alpha_k = \frac{\alpha_0}{1+k}$, with $\alpha_0 = 5$.
   - The evolution of the multiplier λ_t for each timeslot across iterations (in the same plot), for the same cases as in the previous bulletpoint. Additionally plot the evolution of the amount of $\sum_{n \in [1,N]} p_{n,t} - \mathrm{P}^{\mathtt{mall}}$ across iterations, for each t in the same plot.
   - Briefly comment on the results. Also, which stores consume more energy overall? Explain why, both intuitively and also by interpreting how your algorithm behaves (be specific about the mechanics in your own algorithm, not generic).

Credit Allocation Task for Part B.

Report how each group member contributed to Part B.

Deliverable: pdf where

- Each group member writes their name and one paragraph about their own contributions to each Task (around 6-7 sentences in total). Be specific and comprehensive. The group reads, reviews, and approves each member's contributions. Take note that a false statement (for yourself or for others) is a breach of academic integrity.
- The group agrees and jointly provides a number f_i, representing each member's percentage contribution to the entire hand-in (not per Task) - see the General Rules at the top.

  Nuclear option: If you cannot reach an agreement, you can ask for a meeting with the instructor (to be scheduled after your deliverables are evaluated) where each member's contributions and overall understanding of the concepts will be assessed. Use this option only as a last resort; note that it **can** result in reduced points for all members if the assessment reveals weaknesses not discovered through your deliverables alone.

Deadline for Part B: *May 20th, 2026 23.59.*